PROJECT DOCUMENTATION REPORT

EQsono (Sonora)

Multi-Device Synchronized Web Audio Broadcast & Adaptive 5.1 Spatial Surround System

PROJECT TYPE
Full-Stack Web & Audio Sync System

ACADEMIC YEAR
2nd Year (CSE)

INSTITUTION
Aditya University
Submitted as part of 2nd Year undergraduate project coursework (2025–2026 Academic Series).

TEAM MEMBERS & ROLL NUMBERS

S.No.	Student Full Name	Roll Number
1	**Kamepalli Venkata Akhilesh**	25B11CS381
2	**Charaf Kambagiri Kamal Basha**	25B11CS148
3	**Chokka Sowmya Sri**	25B11CS194
4	**Boddu Surya Lakshmi Sahithi**	25B11CS115

DOCUMENT STRUCTURE (6 REQUIRED SECTIONS)

1. **Cover Page** — Project identity & team roster
2. **Overview & Objectives** — Abstract, problem & goals
3. **Technology Stack** — APIs, runtime & architecture
4. **Features & Implementation** — DSP, sync & functions
5. **Screenshots & Results** — Live captures & benchmarks
6. **AI Usage & Contributions** — AI tools & member roles

Repository: EQsono (v1.0.0) Aditya University · Department of CSE November 2026

SECTION 2 Project Overview & Objectives

2.1 Project Abstract

EQsono (Sonora) is a browser-based, multi-device audio synchronization and adaptive spatial sound system designed to convert ordinary smartphones and laptops on a local Wi-Fi or cloud network into a unified speaker array without installing native mobile applications. A host computer captures live system audio, local audio files, or built-in calibration waveforms using Web Audio APIs and an off-main-thread AudioWorklet capture node. Frames are stamped against a shared master timeline calibrated via an RFC 5905-inspired WebSocket NTP probe algorithm with outlier rejection and streamed over low-latency binary WebSockets. In Stereo mode all devices play synchronized audio, while in Surround (5.1) mode a digital signal processing matrix separates Front, Center, 120 Hz Linkwitz-Riley Subwoofer, and decorrelated Rear ambience channels. Adaptive pre-roll buffering, Proportional-Integral clock drift steering, and Packet Loss Concealment maintain tight phase alignment across up to twelve connected devices per isolated room.

Abstract length: 141 words (within the 100–150 word documentation guideline).

2.2 Problem Statement

Everyday group listening, dorm or home movie nights, and ad-hoc collaborative events frequently suffer from weak single-laptop speakers or the high cost of proprietary wireless multi-room and 5.1 surround hardware. While most people in a room already carry smartphones with capable speakers, connecting multiple phones to play the same audio source simultaneously creates severe acoustic challenges:

- **Unsynchronized Playback & Echo:** Standard HTTP audio streaming or naive WebSocket broadcasting introduces variable buffering delays (often 100–800 ms) across heterogeneous phones, turning music or dialogue into an unusable echo.
- **Hardware Clock Drift:** Even if two mobile devices start playback at the same instant, minor oscillator frequency differences and Wi-Fi packet jitter cause audible phase drift over multi-minute tracks.
- **Lack of Spatial Channel Separation:** Replicating stereo streams identically to every device fails to achieve true directional soundstaging (Front Left, Front Right, Center Dialogue, Low-Frequency Subwoofer, and Rear Surround ambience).
- **Friction from Native Installs:** Dedicated speaker-grouping apps require OS-specific downloads, account sign-ins, or proprietary ecosystem lock-in (e.g., vendor-locked smart speakers).

EQsono addresses all four limitations cleanly through standard HTML5/Web Audio APIs served over Express and WebSockets, letting any phone join a studio soundstage in seconds via a QR code scan.

2.3 Main Objectives (4 Core Goals)

OBJECTIVE 1

Sub-Millisecond Distributed Clock Lock

Synchronize the host laptop and up to 12 receiver phones to a common master timeline via lowest-RTT outlier-filtered NTP probes over WebSockets.

OBJECTIVE 2

Low-Jitter Off-Main-Thread Capture & Streaming

Capture stereo frames cleanly inside an AudioWorklet thread (1024-sample quanta) and stream lossless Linear PCM or 4:1 compressed IMA-ADPCM frames.

OBJECTIVE 3

Dual Stereo & Adaptive 5.1 Spatial Matrix DSP

Switch seamlessly between unified Stereo broadcast and discrete 5.1 channel extraction with 120 Hz LFE filtering and Haas rear decorrelation.

OBJECTIVE 4

Zero-Install Onboarding & Cloud/LAN Portability

Enable instant mobile joining via QR/room code with isolated multi-tenant rooms, per-device delay calibration, and one-step Docker/Railway/Render deployment.

2.4 Practical Use Cases

Scenario	Recommended Mode	How EQsono Solves It
Multi-Room / Party Audio	Stereo (Channel `255`)	Up to 12 phones act as synchronized loudspeakers in phase lock with the host laptop's compensated output delay node.
Portable 5.1 Cinema Array	Surround (5.1 Matrix)	Host dynamically assigns devices to FL, FR, Center, LFE (120 Hz lowpass), and SL/SR slots (or lets users map custom roles per phone).
Bandwidth-Constrained Wi-Fi	IMA-ADPCM (4:1)	Reduces each 1024-sample packet payload from 2,064 bytes (PCM) to 532 bytes (ADPCM), staying comfortably beneath the 1,500-byte Ethernet MTU.
Live Acoustic Calibration	Channel Test + Per-Device Delay	Emits isolated channel test tones, web-speech callouts, and mobile haptic pings, while trimming 0–150 ms per-speaker acoustic offsets.

2.5 Operational Design & Platform Constraints

Constraint Area	Design Strategy & Platform Behavior
Browser Secure Context	System audio capture (`getDisplayMedia`) and phone Screen Wake Lock require HTTPS or `localhost`. Cloud hosts supply HTTPS headers automatically; direct LAN deployments support TLS via `SSL_CERT` / `SSL_KEY`, while non-secure LAN sessions warn users gracefully and continue supporting File & Demo sources.
Network Jitter Adaptation	Provides one-click Host Buffer presets: `150 ms` (Studio Wi-Fi), `350 ms` (Public Internet), and `500 ms` (High Jitter / 4G), alongside per-speaker `0–150 ms` hardware/Bluetooth trim sliders.
Room Scalability	Caps rooms at `12` receivers to guarantee real-time binary fan-out budgets on consumer Wi-Fi routers and deploys as a single Node.js instance due to in-memory room state.

2.6 Implemented System Parameters & Thresholds

Parameter	Configured Value	Operational Role in Codebase
Max Receivers / Room	`12` devices	Enforced by `MAX_RECEIVERS_PER_ROOM` in `server.js` (returns `ROOM_FULL`).
Worklet Chunk & Pre-Roll	`1,024` samples / `2` pkts	Off-main-thread capture quantum (~21.3 ms @ 48 kHz) with ~43 ms initial receiver cushion.
Global Buffer Delay Range	`60–600 ms` (default `150`)	Lead scheduling offset covering LAN Wi-Fi (150), WAN (350), and cellular jitter (500).
WS Payload & Heartbeat	`64 KB` max / `15 s` ping	Guards against oversized JSON frames and sweeps stale sockets; idle rooms expire in 30 min.
Clock Steering Limits	`±0.8%` → `±0.15%`	PI playback rate bounds pulling >15 ms drift rapidly before locking inaudibly.

SECTION 3 Technology Stack & System Architecture

EQsono is architected as a lightweight, zero-build web application with strict separation between the Node.js WebSocket relay, browser-native Host DSP capture engine, and mobile Receiver playback scheduler.

3.1 Languages, Frameworks & Core Packages

Layer	Technology / Library	Role in EQsono
Languages	JavaScript (ES2022+), HTML5, CSS3, SVG	Powers Node.js server logic, Web Audio DSP, AudioWorklet processor, accessible semantic views, custom dark studio styles, and inline vector icons.
Server Runtime	Node.js (`v20-alpine` container)	Executes the single-process asynchronous event loop, binary buffer routing, TLS termination option, and OS network interface inspection.
HTTP Framework	Express.js (`^4.19.2`)	Serves static client assets from `public/`, health/info REST endpoints (`/health` , `/api/info` , `/api/room/new`), and proxy-aware receiver routes.
Real-Time Transport	`ws` (`^8.17.0`) + Native Browser WebSocket	Handles bidirectional JSON control signaling, `SYNC_PING` / `SYNC_PONG` timing frames, and zero-overhead binary audio frame forwarding with `TCP_NODELAY` .
Onboarding Utility	`qrcode` (`^1.5.3`)	Generates server-side QR code Data URLs dynamically bound to the detected Wi-Fi IP or reverse-proxied HTTPS cloud hostname and room code.

3.2 State & Database Design Decision

Transient In-Memory Stores + Browser Persistence (Zero External Database Latency): Because live audio frames arrive every ~21 ms (48 kHz / 1024 samples), introducing round-trip SQL/NoSQL database reads during packet relay would degrade jitter performance. Instead, EQsono uses:

- **Server-Side `Map` Stores (`server.js`):** An in-memory `rooms` Map holds transient room state (`roomCode` , sanitized `name` , cryptographic `hostSecret` , connected `receivers` , and `receiverPrefs` retaining roles/delays across brief disconnects). Idle rooms are pruned automatically after 30 minutes.
- **Client `localStorage` Persistence:** Retains host room credentials (`sonora_host_room` , `sonora_host_secret`), preferred codec (`sonora_codec`), buffer delay (`sonora_buffer_delay`), laptop gain (`sonora_laptop_volume`), and persistent mobile speaker ID (`sonora_dev_id`).

3.3 Repository & Deployment Artifact Structure

File / Path	Responsibility
`server.js` , `package.json`	Express HTTP/HTTPS server, room isolation, binary channel router, QR endpoints, and npm dependencies.
`public/index.html` , `sonora-host.js` , `sonora.css`	Host Studio dashboard, 5.1 Soundstage visualizer, accessible dialogs/popovers, and VU transport controls.
`public/receiver.html` , `sonora-receiver.js` , `receiver-sonora.css`	Mobile speaker UI, Web Audio unlock & Wake Lock, jitter queue, PLC concealment, and PI drift locking.
`host-audio.js` , `audio-worklet-processor.js` , `audio-codec.js` , `ntp-sync.js`	Off-main-thread 1024-sample capture, 5.1 DSP matrix, 4:1 IMA-ADPCM codec, and RFC 5905 NTP client.
`Dockerfile` , `railway.json` , `render.yaml`	Production Alpine container image and cloud PaaS deployment manifests.

3.4 Browser APIs, Security & Deployment Tooling

Category	APIs / Tools Used	Purpose
Web Audio & DSP	`AudioContext`, `AudioWorkletNode`, `ScriptProcessorNode` (fallback), `AnalyserNode`, `DelayNode`, `GainNode`	Captures off-main-thread 1024-sample stereo quanta, compensates host output latency, drives live FFT bars, and schedules phase-locked playback on phones.
Media & Hardware	`getDisplayMedia()`, HTML5 `Audio` + Blob URLs, `navigator.wakeLock`, `navigator.vibrate`, `SpeechSynthesis`	Captures laptop system/tab audio, decodes dropped/uploaded media files, keeps mobile screens awake during playback, vibrates on Ping, and announces test channels.
High-Res Timing	`performance.timeOrigin + performance.now()`	Provides sub-millisecond epoch timestamps on both Node.js server and browser clients for NTP clock synchronization and jitter measurement.
Modern UI & DOM	Native `<dialog closedby="any">`, HTML `popover="auto"`, DOM-only text binding	Provides accessible modal management with Safari pointer-dismiss fallback, lightweight room controls, and XSS-safe rendering without unsafe HTML injection.
Security & Crypto	`crypto.randomBytes`, `crypto.timingSafeEqual`, Node `https` (optional `SSL_CERT` / `SSL_KEY`)	Prevents unauthorized host takeover via secret tokens, strips control characters/tags from client text, enforces 64 KB WS payload caps, and supports direct LAN TLS.
DevOps & Testing	Docker (`Dockerfile`), `railway.json`, `render.yaml`, Headless Chrome CDP	Packages production non-root containers, configures PaaS deployment, and executes automated multi-tab E2E audio/UI/security regression suites.

3.5 High-Level System Architecture

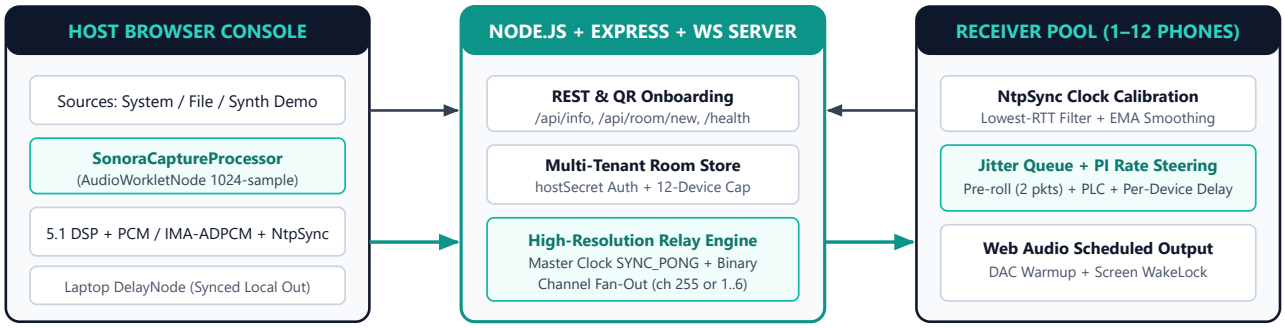


Figure 3.1: High-level distributed audio capture, master-clock relay, and synchronized receiver playback architecture.

SECTION 4 Features & Implementation

4.1 Key Features & End-to-End Application Workflow

EQsono combines consumer-friendly setup with real-time broadcast tooling across two synchronized web clients:

- **Multi-Source Host Capture:** Captures system/tab audio via `getDisplayMedia()` (disabling voice-processing filters like echo cancellation and AGC for pure music fidelity), plays drag-and-dropped/uploaded media files with seek/pause/resume/loop controls, or generates three built-in calibration demos (*Synthwave 80s*, *5.1 Orbit*, and *Sync Pulse*).
- **Dynamic Room Isolation & QR Pairing:** Assigns a random 6-character alphanumeric room code paired with a 128-bit `hostSecret` so multiple rooms run cleanly on one server without cross-room interference or hijacking.
- **Stereo vs. Adaptive 5.1 Placement:** Automatically distributes 1–6+ joined speakers around the visual Soundstage (`FRONT_LEFT`, `FRONT_RIGHT`, `CENTER`, `SUBWOOFER`, `REAR_LEFT`, `REAR_RIGHT`) while allowing manual per-device override and selective server channel routing to conserve Wi-Fi bandwidth.
- **Acoustic Calibration Suite:** Combines a global `60–600 ms` broadcast buffer delay (with Wi-Fi / WAN / 4G presets), `0–150 ms` per-speaker delay trim, tactile device Ping (vibration + flash), and channel-isolated test tones with spoken callouts.

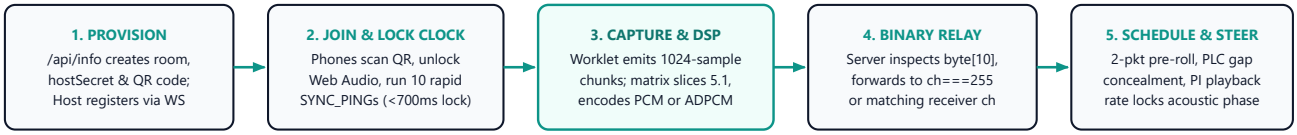


Figure 4.1: End-to-end EQsono connection, capture, binary forwarding, and receiver playback workflow.

4.2 Binary Packet Format & Audio Compression

Every live audio quantum is transmitted in a tight little-endian binary WebSocket frame with a fixed 16-byte header prepended to the audio samples, avoiding JSON parsing overhead on high-frequency streaming threads:



Figure 4.2: Little-endian 16-byte EQsono wire packet header layout for 1024-sample capture chunks.

In lossless **Linear PCM** mode (`flags = 0x00`), 1,024 16-bit samples produce 2,048 payload bytes (**2,064 bytes total** per channel packet). When switched to **IMA-ADPCM** (`flags = 0x01`), `AudioCodec.encodeImaAdpcm()` writes a 4-byte state header (initial `Int16` sample, initial `stepTable` index seeded from the first sample delta, reserved byte) followed by packed 4-bit nibbles — cutting total packet size to **532 bytes** (~74% reduction) so frames transmit comfortably inside a single 1,500-byte Ethernet MTU without IP fragmentation.

4.3 Backend Relay & Synchronization Subsystems

Server Room Management & Binary Router (`server.js`)

- `createRoom()` & `getOrCreateRoom()` : Generates unique 6-character codes (`ABCDEFGHJKLMNPQRSTUVWXYZ23456789`) paired with a 32-hex-char `hostSecret` . Re-fetching `/api/info` only returns an existing room if verified via constant-time `crypto.timingSafeEqual()` inside `secretsEqual()` ; unauthorized or stale requests create an isolated new room.
- `sanitizeText()` : Strips control characters and angle brackets (`<>`) and bounds lengths before saving or relaying room names, device names, roles, or colors.
- `cleanupSocketMembership()` & `notifyHostDeviceList()` : Cleans host or receiver bindings immediately upon close/error, emits `HOST_STATUS` transitions to receivers, preserves `receiverPrefs` across temporary drops, and throttles frequent UI list updates (800 ms window unless forced).
- `wss.on('message')` Binary Path: Reads `message.readUInt8(10)` without decoding audio payloads, routes only to matching open sockets where `targetChannel === 255 || receiver.channelIndex === targetChannel` , and enforces a `512,000` -byte `bufferedAmount` backpressure threshold per receiver so one slow phone never stalls other listeners.
- **Lifecycle & Proxy Support:** Enables `app.set('trust proxy', true)` for Render/Railway HTTPS URL detection, runs 15-second WebSocket ping/pong liveness sweeps, prunes empty rooms after 30 minutes idle, exposes `GET /health` , optionally boots Node HTTPS via `SSL_CERT` / `SSL_KEY` , and handles `SIGTERM` / `SIGINT` via `gracefulShutdown()` .

High-Precision Clock Alignment (`NtpSync` in `public/js/ntp-sync.js`)

Immediately on WebSocket open, `NtpSync.start()` fires a rapid burst of 10 `SYNC_PING` messages spaced 70 ms apart (achieving lock in under 700 ms) before transitioning to maintenance pings every 2 seconds. For each `SYNC_PONG` , using client transmit time `t0` , server receive time `t1` , server transmit time `t2` , and client receive time `t3` :

```
Round-Trip Time: rtt = max(0, (t3 - t0) - max(0, t2 - t1))
RFC 5905 Offset: offset = ((t1 - t0) + (t2 - t3)) / 2   |   Master Clock: now() = localTime + offset
```

To reject asymmetrical Wi-Fi queue spikes, `handlePong()` retains a rolling window of 20 samples, sorts by lowest RTT, selects the fastest **45%** (`bestSamples`), computes their median offset, and applies an exponential moving average (`0.85 * old + 0.15 * medianOffset` ; `0.75 * old + 0.25 * dev` for jitter). Both `this.rtt` and jitter derive strictly from low-RTT samples, driving stable UI tiers: `STUDIO_LOCK` (<18 ms RTT, ≤3 ms jitter), `PRECISION` (<40 ms RTT, ≤8 ms jitter), or `GOOD` .

4.4 Host DSP Engine, Spatial Controller & Receiver Scheduler

Off-Main-Thread Capture & 5.1 Matrix DSP (`host-audio.js` & `audio-worklet-processor.js`)

- `SonoraCaptureProcessor` : Runs inside the Web Audio rendering thread, zeroing output buffers to stop un-delayed bleed and packing 128-sample Web Audio quanta into zero-copy transferable 1024-sample stereo buffers with exact `currentTime` capture stamps. Non-secure LAN HTTP environments cleanly fall back to `createScriptProcessor(1024, 2, 2)` .
- `HostAudioEngine.handlePcmFrame()` : Maintains a continuous sample counter anchored at initial capture time (accounting for capture age `ageMs` and live `ntpSync.offset`) so packet playback target timestamps advance by exact sample counts rather than noisy JS callback jitter. Any >150 ms discontinuity triggers smooth re-anchoring.
- **Surround 5.1 Matrix Math:** In Surround mode, `handlePcmFrame()` computes only live assigned channels: Center (`(L+R)*0.65` , conditionally subtracted from FL/FR), LFE (120 Hz Linkwitz-Riley lowpass via `_setupSubFilter()` , Q=0.7071), and Rear SL/SR (ambience through a `c=0.6` all-pass shifter + 15 ms Haas delay with inverted polarity), all clamped via `_softLimit(x)` above `0.7` .
- `playChannelTest()` & `streamChannelTestTone()` : Synthesizes local monitor oscillator callouts plus Web Speech names alongside deterministic 0.6 s enveloped PCM test packets (60 Hz LFE / 660 Hz directional channels) streamed cleanly to the target receiver while keeping Ping feedback visually/haptically silent on duplicate tones (`silentAudio: true`).

Host UI & Adaptive Placement (`sonora-host.js`)

- `init()` awaits `loadServerInfo()` before opening WebSockets so room tokens always exist prior to registration, displays an informational banner when opened over plain HTTP LAN without `isSecureContext` , and restores persisted buffer delay, codec, and laptop speaker gain from `localStorage` .
- `applyAdaptiveSurroundRoles()` & `updateStageSlots()` : Assigns 1–6+ phones around optimal spatial coordinates (or honors `deviceCustomRoles`), updates all 6 radar slots, and passes active channels to `audioEngine.setActiveChannels()` .
- **XSS-Safe Incremental DOM Updates:** Builds speaker rows via `document.createElement` and `textContent` , keyed by a render signature so `updateSpeakerTelemetry()` patches RTT/offset live without resetting delay sliders or dropdowns.

Mobile Receiver Unlock, Jitter Buffer & PI Rate Steering (`sonora-receiver.js`)

- `unlockAudio()` : Responds to user tap on mobile browsers by creating an `AudioContext({ latencyHint: 'interactive' })` , firing a 1-sample silent warmup buffer plus a `0.00001` gain keepalive oscillator (preventing mobile hardware DAC sleep), requesting `navigator.wakeLock.request('screen')` , starting a 50 ms queue drain timer, and converting subsequent taps into instant clean mute/unmute toggles.
- `handleAudioPacket()` : Filters incoming binary frames strictly to channel `255` or matching `myChannelIndex` , decodes PCM or ADPCM, caps queue growth at 25 packets to prevent runaway latency, and performs exponential-decay **Packet Loss Concealment (PLC)** synthesizing a bridge frame whenever a single sequence number gap (`lastSeq + 2`) occurs.
- `drainAudioQueue()` : Buffers `PRE_ROLL_PACKETS = 2` chunks (~43 ms), prunes stale overdue frames, applies a 64-sample cosine fade-in (`applyBoundaryFadeIn()`) on stream starts/underruns, schedules consecutive `AudioBufferSourceNode` instances sample-accurately via `scheduledUntil` , and drives a bounded **Proportional-Integral (PI) clock-steering loop** adjusting `playbackRate` by up to `±0.8%` for drift >15 ms before settling to an inaudible `±0.15%` micro-lock band.

4.5 WebSocket Control Protocol Summary

Message Type	Direction	Key Payload & Function
`SYNC_PING` / `SYNC_PONG`	Client ↔ Server	Exchanges `clientSendTime` , `serverReceiveTime` , and `serverSendTime` for RFC 5905 offset calibration.
`REGISTER`	Host/Recv → Server	Authenticates Host (`roomCode` + `hostSecret`) or registers Receiver (`deviceId` , sanitized model `info`).
`HOST_REGISTERED` / `HOST_REJECTED` / `HOST_REPLACED`	Server → Host	Confirms room binding, clears stale credentials on restart, or cleanly hands over if a newer host tab opens.
`REGISTERED` / `JOIN_REJECTED` / `ROOM_FULL`	Server → Receiver	Confirms room join, restores saved role/delay prefs and `hostOnline` state, or rejects full (≥12) / invalid rooms.
`DEVICE_LIST` & `TELEMETRY`	Recv → Srv → Host	Pushes per-device RTT/offset up to the Host every ping cycle (throttled broadcast at 800 ms intervals).
`SET_MODE` / `ASSIGN_ROLE` / `SET_SPEAKER_DELAY` / `PING_DEVICE`	Host → Srv → Recv	Switches `PARTY` / `DOLBY` mode, assigns spatial channel index, calibrates 0–500 ms receiver delay, and triggers haptics/visual ping.

4.6 Reliability, Fault Recovery & Security Controls

- **Host Credential Recovery & Takeover:** Opening a duplicate host tab sends `HOST_REPLACED` (close code `4001`) to the prior session, while stale secrets after a server restart return `HOST_REJECTED` so the client transparently provisions a fresh room.
- **Receiver Resilience:** Drops trigger automatic 2-second reconnection loops, restore assigned spatial roles and calibrated delays from `room.receiverPrefs` , update offline status via `HOST_STATUS` , and re-arm pre-roll + Screen Wake Lock on tab `visibilitychange` .
- **Backpressure & XSS Defense:** Server skips binary transmission whenever a socket exceeds `512,000` buffered bytes; host DOM construction exclusively uses `document.createElement` and `textContent` paired with server-side `sanitizeText()` filtering.

SECTION 5 Screenshots & Verification Results

Figures 5.1 through 5.5 show real captures from the running EQsono Node.js server paired with a desktop Host (`1280px`) and three isolated mobile Receiver sessions (`390×844`) running simultaneously over Chrome Web Audio and WebSockets.

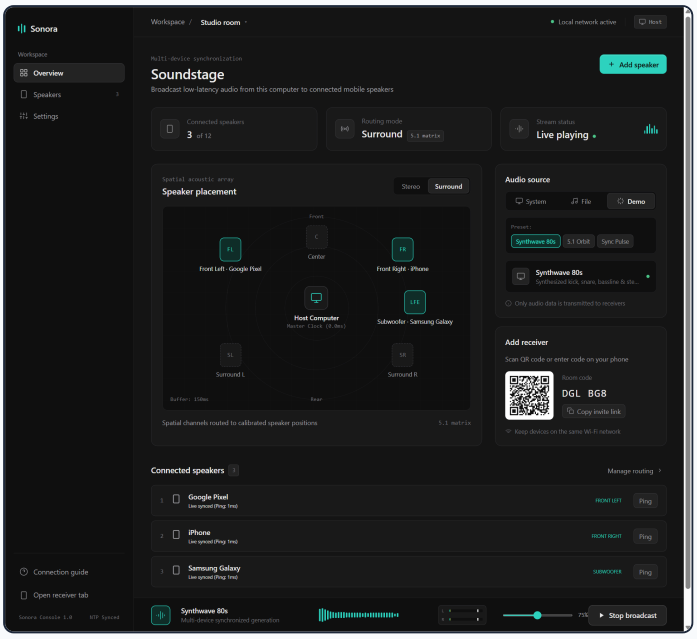


Figure 5.1 — Host Soundstage Overview (Surround 5.1 Active Broadcast): Displays three connected devices mapped spatially on the Soundstage (`Google Pixel` on Front Left, `iPhone` on Front Right, and `Samsung Galaxy` on Subwoofer LFE), live *Synthwave 80s* demo generation, dual stereo LED meters, animated transport waveform, QR room pairing card, and real-time connected speaker statuses.

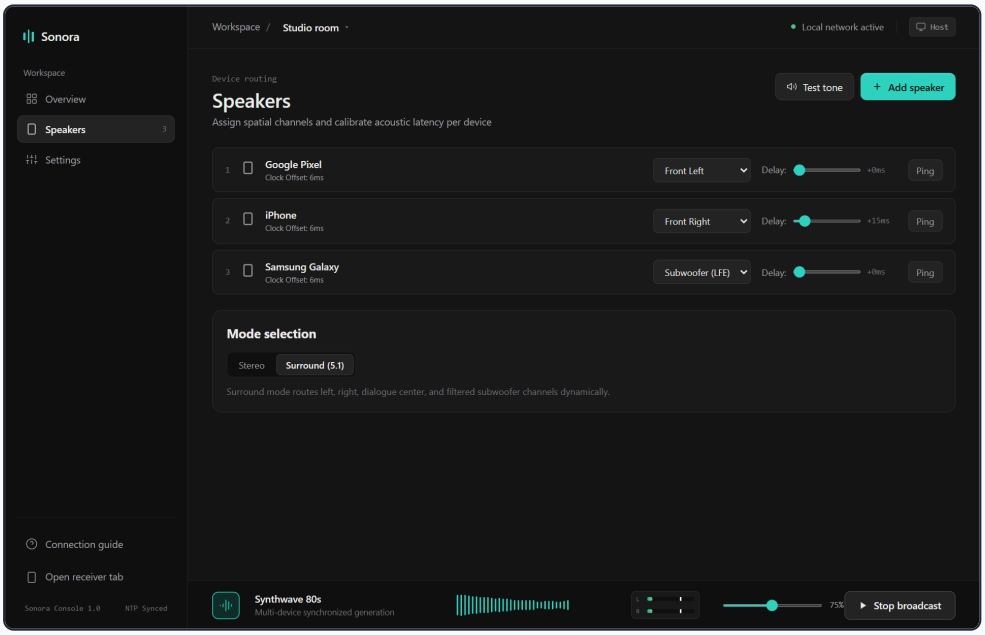


Figure 5.2 — Speakers Management & Acoustic Delay Calibration: Illustrates per-device WebSocket clock offsets (6 ms), interactive spatial role dropdown selectors, calibrated +15 ms acoustic delay slider offset applied to the iPhone receiver, Ping locate triggers, and global Stereo / Surround (5.1) mode controls.

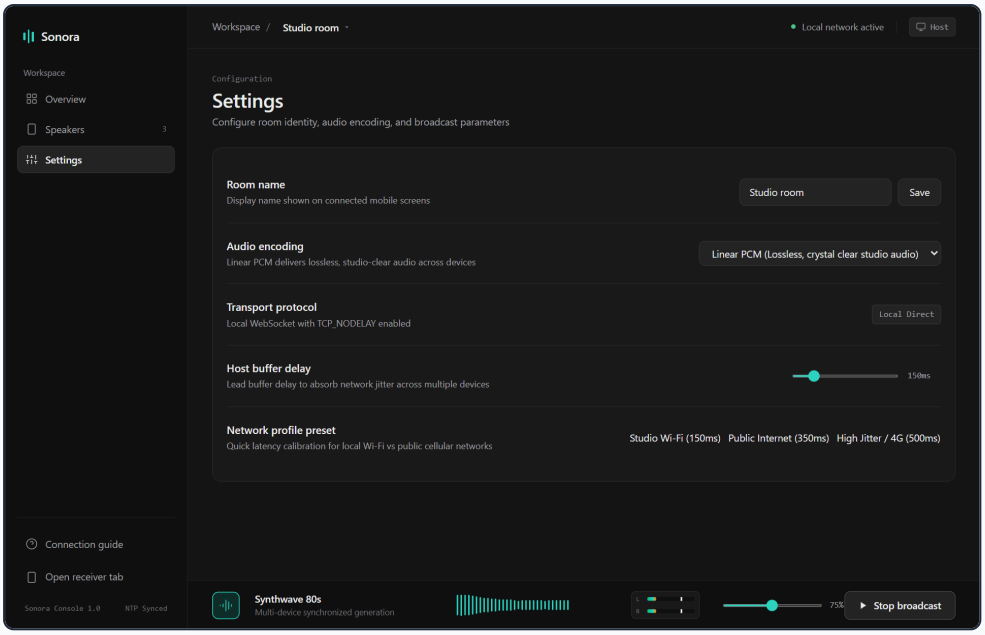


Figure 5.3 — Broadcast & Network Settings View: Shows live room display renaming, runtime toggling between lossless Linear PCM and 4:1 IMA-ADPCM encoding, Local Direct WebSocket transport with `TCP_NODELAY`, customizable lead buffer delay (150 ms default), and one-click Studio Wi-Fi / Public Internet / High Jitter 4G presets.

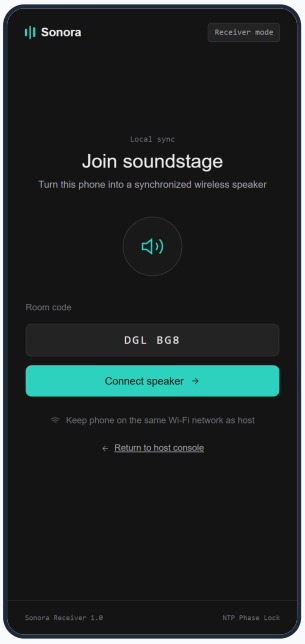


Figure 5.4 — Mobile Receiver Onboarding (390×844): Zero-install mobile join view pre-filled or entered via QR room code, built with unconstrained accessibility zoom and clear same-network guidance.

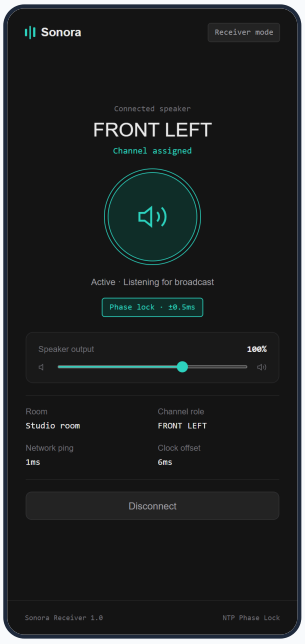


Figure 5.5 — Active Phase-Locked Mobile Speaker: Shows assigned `FRONT LEFT` role, `Phase lock · ±0.2ms` badge, tactile mute/unmute aura ring, live 100% gain slider, 1 ms network ping, and 3 ms clock offset telemetry.

5.6 Automated Verification & Audio Streaming Results

Prior to documentation sign-off, an automated Headless Chrome CDP + Node WebSocket verification suite tested syntax integrity, authentication isolation, mobile layout overflow, live binary packet timing, file playback controls, and XSS safety against a live `node server.js` instance:

Verification Check	Measured Result	Significance
Syntax & Runtime Errors	7/7 JS files passed `node -c`; `0` host/receiver console or runtime errors; `0` HTTP ≥400 failures	Confirms clean static syntax, zero dead script references, and error-free client page loads.
AudioWorklet PCM Stream (1.2s)	`55` packets, `2,064 B/pkt`, `48,000 Hz`, `0` sequence gaps; headroom `130 ms` → `125 ms`	Confirms zero dropped/reordered frames on localhost with strong scheduling margin before target playback timestamps.
Live IMA-ADPCM Switch	`47` packets at `532 B/pkt` (`flags = 1`)	Demonstrates ~74% bandwidth reduction dynamically mid-stream without tearing down rooms.
File Drop & Transport Controls	WAV drop auto-loaded; Pause → `Resume broadcast`, Loop toggle → `Loop: Off`, Resume → `Pause broadcast`	Verifies non-destructive file pausing, seek/loop state tracking, and clean transport toggling.
Surround Channel Test Isolation	`29` ch-1 packets routed to assigned ch-1 listener; ch-2 packet isolated; `silentAudio: true` ping	Confirms multi-channel isolation and non-overlapping test tones during speaker calibration.
Security, Capacity & Responsive	`HOST_REJECTED` on wrong secret; 13th device → `ROOM_FULL`; XSS payload neutralized (`fired: false`); `0` overflow at `390x844`	Validates room isolation, 12-device cap enforcement, DOM escaping, and mobile layout readiness.

Production Deployment Scope Note: Automated runs validated synthetic Demo streams, dropped WAV media, Chrome mobile emulation, and Node packet timing on localhost. System Audio desktop sharing (`getDisplayMedia` OS dialog) requires interactive user confirmation, and production cloud hosts require HTTPS (provided automatically by Render/Railway) for Wake Lock and screen/tab capture APIs, running as a single instance due to in-memory room state.

5.7 Measured Wire Codec Comparison (1,024-Sample Worklet Frames @ 48 kHz)

Encoding Mode	Header + Payload	Frame Flag	Observed Network & Acoustic Trade-off
Linear PCM (Default)	`2,064 B/pkt`	`0x00`	Bit-exact 16-bit lossless samples (~46.9 packets/s @ 48 kHz), ideal for home Wi-Fi hi-fi listening.
IMA-ADPCM (4:1)	`532 B/pkt`	`0x01`	~74% smaller frame fitting cleanly inside a single 1,500 B Ethernet MTU with sub-percent sample quantization error.

SECTION 6 AI Usage & Team Contributions

6.1 AI Tools Used & Purpose

During the refinement, optimization, verification, and technical documentation phases of EQsono, the team utilized **Antigravity (AI Coding Assistant)** in a supervised engineering workflow. All architectural decisions, verification scripts, and source edits were reviewed and tested directly against the live Node.js server and Chrome DevTools environments.

AI Tool / Phase	Targeted System Area	Specific Engineering & Documentation Purpose
Antigravity *Security & Bug Audit*	`server.js`, `sonora-host.js`, `sonora-receiver.js`	Identified and fixed missing `hostSecret` room authorization, `innerHTML` XSS vector in speaker rows, initialization race ordering, stale socket cleanup, and WebSocket malformed-frame safeguards.
Antigravity *Audio DSP & Sync Upgrade*	`audio-worklet-processor.js`, `host-audio.js`, `ntp-sync.js`	Implemented off-main-thread `SonoraCaptureProcessor` capture, fixed surround circular buffer sample-rate sizing, smoothed RTT/jitter outlier tracking, improved PI drift locking, and added file pause/seek/loop controls.
Antigravity *UI / Accessibility Polish*	`index.html`, `receiver.html`, CSS stylesheets	Migrated custom modals to native `<dialog closedby="any">` and `[popover]` menus, restored mobile viewport zoom, removed render-blocking font imports, eliminated mobile horizontal overflow, and removed dead legacy files.
Antigravity *Verification & Docs*	Headless Chrome CDP & PDF Report	Authored end-to-end automated regression scripts, verified PCM/ADPCM streaming latency headroom, captured live multi-device screenshots, and compiled this structured documentation report.

6.2 Team Member Contributions

EQsono was developed collaboratively by the four-member 2nd Year CSE team at Aditya University, divided across backend relay networking, digital audio processing, synchronized mobile playback, and responsive dashboard engineering:

Team Member & Roll No.	Primary Module Area	Key Project Responsibilities
Kamepalli Venkata Akhilesh `25B11CS381`	Backend Relay, Room Security & Deployment	Designed Express HTTP/HTTPS server, dynamic multi-tenant room creation and constant-time secret verification, binary channel router, idle cleanup, healthcheck endpoint, and Docker/Railway/Render configs (`server.js`, `Dockerfile`).
Charaf Kambagiri Kamal Basha `25B11CS148`	Host Audio Engine, Worklet & DSP Matrix	Engineered `HostAudioEngine` capture graphs, `AudioWorkletProcessor` stereo frame chunking, 5.1 surround matrix extraction with 120 Hz LFE and Haas ambience filters, IMA-ADPCM compression, and built-in Demo/Test generators.
Chokka Sowmya Sri `25B11CS194`	NTP Clock Sync & Mobile Receiver Engine	Built RFC 5905 `NtpSync` lowest-RTT calibration, mobile Web Audio unlock & Wake Lock lifecycle, jitter buffer pre-roll, single-packet PLC bridge concealment, PI clock-rate steering, and responsive receiver interface (`receiver.html`).
Boddu Surya Lakshmi Sahithi `25B11CS115`	Host Dashboard UI, Soundstage & Verification	Developed Host Soundstage visualizer, adaptive role assignment logic, QR onboarding & network interface switcher, native accessible dialog/popover UX, CSS responsive themes, and integration/end-to-end verification coordination.

Note: All four members collaborated jointly on problem definition, architecture design, multi-device integration checks, and final documentation review.